Лабораторная работа №2

Верификация модели турникета метро в SPIN

Касимов Владислав Андреевич

2026

1. Постановка задачи
2. Модель на Promela
3. Свойства на LTL
4. Результаты верификации
5. Нетривиальность посылок
6. Анализ невыполненного свойства P4
7. Сравнение nuXmv и SPIN
Приложение A. Модель `turnstile.pml`

1. Постановка задачи

Систему из ЛР1 (турникет в метро, вариант 2) нужно описать на Promela, сформулировать три свойства на LTL, проверить их в SPIN и сравнить подходы SPIN и nuXmv. Требования к системе и к свойствам те же, что в ЛР1.

Описание системы и смысл свойств не изменились, см. ЛР1, раздел 2. Здесь подробно разобрано только то, что меняется при переходе к асинхронной модели.

2. Модель на Promela

Процессы и взаимодействие

Каждый компонент описан отдельным `active proctype`. Всего восемь процессов, и SPIN выполняет их с чередованием: на каждом шаге выполняется один оператор одного процесса.

```
 Passenger --card_in [0]--> CardReader    --\
     |      --token_in[0]--> TokenAcceptor --+--to_ctrl [2]--> Controller --gate_cmd[1]--> Gate
     |                                        /                       \----lamp_cmd[1]--> Lamp
     +-- beam, beam_hit --> PassSensor -----/
 Intruder --(читает in_passage, меняет beam)
```

Канал	Ёмкость	Назначение
`card_in`, `token_in`	0 (рандеву)	пассажир передаёт носитель датчику. Носитель передаётся физически, поэтому подходит синхронная передача
`to_ctrl`	2	события датчиков `PAID`, `REJECTED`, `PASSED`, `TAKEN` для контроллера
`gate_cmd`, `lamp_cmd`	1	команды актуаторам

Общие переменные `gate`, `lamp`, `user_st`, `ctrl_st`, `card_res`, `token_res` моделируют наблюдаемое состояние. По ним формулируются LTL-свойства. Переменная `beam` — счётчик людей в проходе. Фотоэлемент видит только `beam > 0` и через защёлку `beam_hit` выдаёт импульс `PASSED`, когда проход освободился.

Отличия от модели nuXmv

- **Асинхронность.** В ЛР1 все модули шагали одновременно, здесь шаги процессов чередуются. Между командой `gate_cmd!OPEN` и фактическим `gate = OPEN` могут выполниться любые шаги других процессов.
- **Обмен сообщениями.** Контроллер не опрашивает переменные датчиков, а получает события из очереди. Последовательный алгоритм (ждать `PASSED` → закрыть → дождаться закрытия → погасить табло) записывается прямо как последовательность операторов. В nuXmv для этого пришлось бы вводить отдельные состояния.
- **Неделимость.** Блоки `atomic` моделируют физически одновременные события. Например, когда пассажир входит в проход, луч перекрывается в тот же момент.

- **Безбилетник** ждёт, когда в проходе появится оплативший пассажир (`in_passage`), и проскакивает за ним.

Полный текст модели приведён в приложении А.

3. Свойства на LTL

Свойства подключаются по одному через препроцессор (`spin -DP1 -a turnstile.pml`), поэтому для каждого собирается отдельный верификатор `pan`.

№	Формула Promela	Структура
P1	`[] (rejected -> (gate == CLOSED U lamp == RED))`	безопасность с Until
P2	`[] (paid -> <> (user_st == WALK))`	отклик
P3	`[] ((gate == CLOSED && X (gate == OPEN)) -> X (gate == OPEN U (beam > 0)))`	X и Until
P4	`[] !sneaked`	инвариант (**ожидается нарушение**)
L1	`[] <> (ctrl_st == IDLE && gate == CLOSED)`	живость: система бесконечно часто возвращается в исходное состояние

P1 в nuXmv было CTL-формулой `AG(… → A[·U·])`. SPIN понимает только LTL, поэтому здесь записан LTL-аналог. Для этого свойства он эквивалентен: `A[φ U ψ]` под `AG` выражает то же самое, что `G(p → φ U ψ)` по всем путям.

Оператор `X` и редукция частичных порядков

Редукция частичных порядков (POR) сохраняет только свойства, инвариантные к заиканию, а `X` такой инвариантностью не обладает: вставка шага другого процесса меняет «следующее» состояние. Поэтому верификаторы `pan` собираются с `-DNOREDUCE`.

4. Результаты верификации

Каждое свойство проверяется отдельно: `bash lab2-spin/run.sh`. Флаги `pan`: `-a` включает поиск циклов допуска (для LTL), `-f` — слабую справедливость, `-m200000` задаёт глубину поиска. Компиляция: `gcc -O2 -DNOREDUCE`. Проверка SAFETY без LTL ищет тупики (*invalid end states*) и нарушения `assert`, собирается с `-DSAFETY`.

Проверка	Флаги	Состояний	Переходов	Глубина	Результат
SAFETY (тупики)	—	2504	7132	367	**0 errors**
P1	`-a`	2958	10742	722	**0 errors**
P2	`-a`	3114	11692	722	**0 errors**
P3	`-a`	4876	14195	722	**0 errors**

Проверка	Флаги	Состояний	Переходов	Глубина	Результат
P4	`-a`	23	24	43	**1 error** (assertion violated в never claim)
L1	`-a -f`	4792	78685	785	**0 errors**

Каждый прогон занимает около 0.05 с.

Число состояний больше, чем в nuXmv (108), в 23–45 раз. Причина в том, что в состояние SPIN входят счётчики команд всех восьми процессов и содержимое каналов. Промежуточные точки последовательного кода контроллера становятся отдельными состояниями, а в синхронной модели их схлопывает один такт. Кроме того, в число состояний входит состояние автомата Бюхи из never claim, поэтому для разных свойств оно разное.

Справедливость

Для P1–P3 справедливость не нужна. Для L1 нужна: без `-f` SPIN находит цикл, в котором планировщик бесконечно выбирает пассажира с веткой «никто не подошёл», а контроллер, готовый вернуться в `IDLE`, так и не получает ход. Это несправедливое расписание. Со слабой справедливостью (`-f`) L1 выполняется.

5. Нетривиальность посылок

Подход здесь иной, чем в nuXmv. SPIN проверяет только LTL, то есть «для всех путей», и не умеет сказать «существует путь». Поэтому проверяется *отрицание* свойства «посылка выполняется бесконечно часто». Найденная ошибка и есть искомый свидетель: лассо, где посылка повторяется на каждом обороте цикла.

Проверка	Формула	Посылка	Состояний	Результат
V1	`!([] <> rejected)`	P1	262	**1 error**: найден цикл с бесконечно частым отказом
V2	`!([] <> paid)`	P2	133	**1 error**: цикл с бесконечно частой оплатой
V3	`!([] <> (gate == CLOSED && X (gate == OPEN)))`	P3	226	**1 error**: створки открываются бесконечно часто

6. Анализ невыполненного свойства P4

Трасса (`spin -DP4 -t -p turnstile.pml`) коротко, глубина 43:

Шаг	Процесс	Оператор	Пояснение
2–10	Passenger	`user_st = APPROACH; m = CARD_OK; card_in!m`	пассажир приложил действующую карту
11–17	CardReader	`card_res = OK; to_ctrl!PAID`	валидатор подтвердил оплату

Шаг	Процесс	Оператор	Пояснение
19–23	Controller	`to_ctrl?msg; … ctrl_st = GRANTED`	контроллер принял оплату
25–29	Controller, Lamp	`lamp_cmd!GREEN` → `lamp = GREEN`	зелёный
31–35	Controller, Gate	`gate_cmd!OPEN` → `gate = OPEN`	створки открыты
37–41	Passenger	`user_st = WALK; beam++; beam_hit = 1; in_passage = 1`	пассажир вошёл в проход, луч перекрыт
43	**Intruder**	`in_passage → beam++; beam_hit = 1; sneaked = 1`	**безбилетник проскочил следом: нарушение**

Причина та же, что в nuXmv. Луч перекрыт с момента входа пассажира, безбилетник добавляет к нему второе перекрытие, и `beam` уходит в 0 один раз. `PassSensor` выдаёт один импульс `PASSED`, и контроллер закрывает створки, считая, что прошёл один человек. Счётчик `beam` в модели есть, но фотоэлемент его не видит: он видит только `beam > 0`, как реальный однолучевой датчик.

Обоснование, почему исправление неприемлемо сложно, приведено в ЛР1, раздел 7. Нужен датчик, различающий людей, а также геометрия прохода и временные интервалы в модели. Кроме того, закрывать створки раньше, чем перекрыт луч, нельзя: это нарушит P3. В асинхронной модели к этому добавляется гонка. Даже если бы контроллер знал число людей, между `beam++` безбилетника и реакцией контроллера на событие сколько угодно шагов. Поэтому ограничивать проход пришлось бы механически (трипода, шлюз), а не программно.

7. Сравнение nuXmv и SPIN

Аспект	nuXmv (ЛР1)	SPIN (ЛР2)
Модель вычислений	синхронная композиция модулей, все делают шаг одновременно	асинхронное чередование процессов, по одному оператору за шаг
Взаимодействие	общие переменные, реакция на следующем такте	каналы (рандеву и буферизованные) и общие переменные
Описание поведения	функции `next(x) := case …`, явный автомат	последовательный императивный код с `do`/`if`/`atomic`
Логики	CTL и LTL (и PSL)	только LTL; CTL-свойства нужно переводить в LTL или проверять через отрицание
`X`	без ограничений	несовместим с POR, нужен `-DNOREDUCE`
Справедливость	не нужна для синхронной модели (`FAIRNESS` есть)	слабая справедливость по процессам (`-f`); для L1 обязательна
Метод проверки	символьный (BDD), строит множество достижимых состояний целиком	явный перебор в глубину (DFS), never claim из отрицания формулы, вложенный DFS для циклов

Аспект	nuXmv (ЛР1)	SPIN (ЛР2)
Размер пространства	108 состояний	2504 без свойства, 2958–4876 с never claim
Время	0.02 с на все свойства	≈0.05 с на свойство
Контрпример	последовательность состояний, лассо помечено «Loop starts here»	трасса по процессам и операторам (`spin -t -p`), начало цикла `<<<<<START OF CYCLE>>>>>`
«Существует путь»	`EF`, `EG` напрямую	только как нарушение отрицания

Выводы о методах. nuXmv удобнее для систем, которые естественно описываются тактами: аппаратура, PLC-подобные контроллеры. Модель получается компактнее, есть CTL, справедливость обычно не нужна. SPIN ближе к программе: протоколы, обмен сообщениями, последовательные алгоритмы. Цикл контроллера здесь читается как обычный код. Цена этого — большее пространство состояний, необходимость думать о справедливости и об инвариантности к заиканию (`X` против POR), а также только LTL.

На этой модели оба инструмента дали одинаковые выводы. P1–P3 выполняются, P4 нарушается тем же сценарием «паровозика». Это подтверждает, что результат определяется устройством системы (одним фотоэлементом), а не выбором семантики.

Приложение А. Модель `turnstile.pml`

```
/* ========================================================================
 * Лабораторная работа №2. Турникет в метро (вариант 2), язык Promela.
 * Асинхронная модель: процессы чередуются, датчики общаются с
 * контроллером через буферизованный канал, контроллер управляет
 * актуаторами через каналы команд.
 *
 * Процессы: Passenger, Intruder (среда), CardReader, TokenAcceptor,
 *           PassSensor (датчики), Controller (управление),
 *           Gate, Lamp (актуаторы).
 *
 * Свойства подключаются препроцессором: spin -DP1 -a turnstile.pml
 * ======================================================================== */

mtype = {
  NONE, OK, FAIL,                              /* результат датчика оплаты */
  CARD_OK, CARD_EMPTY, TOKEN_OK, TOKEN_BAD,    /* носитель у пассажира     */
  TAKE,                                        /* пассажир забрал носитель */
  PAID, REJECTED, PASSED, TAKEN,               /* сообщения контроллеру    */
  OPEN, CLOSED,                                /* створки (и команды)      */
  OFF, GREEN, RED,                             /* табло (и команды)        */
  AWAY, APPROACH, WAIT, WALK,                  /* состояния пассажира      */
  IDLE, GRANTED, DENIED, CLOSING               /* состояния контроллера    */
};

chan card_in  = [0] of { mtype };   /* пассажир -> валидатор (рандеву) */
chan token_in = [0] of { mtype };   /* пассажир -> приёмник жетонов    */
chan to_ctrl  = [2] of { mtype };   /* датчики -> контроллер           */
chan gate_cmd = [1] of { mtype };   /* контроллер -> створки           */
chan lamp_cmd = [1] of { mtype };   /* контроллер -> табло             */

mtype gate = CLOSED, lamp = OFF;
mtype user_st = AWAY, ctrl_st = IDLE;
mtype card_res = NONE, token_res = NONE;  /* результат, ещё не забранный контроллером */
byte  beam = 0;            /* сколько людей сейчас в проходе (фотоэлемент) */
bool  beam_hit = false;    /* защёлка датчика: луч перекрывался */
bool  in_passage = false;  /* оплативший пассажир в проходе */
bool  sneaked = false;     /* безбилетник в проходе (для P4) */

#define paid     (card_res == OK   || token_res == OK)
#define rejected (card_res == FAIL || token_res == FAIL)

/* ---------------------------- среда ---------------------------- */
active proctype Passenger() {
  mtype m;
  do
  :: (gate == CLOSED && lamp == OFF) -> skip    /* никто не подошёл (охранник: SPIN запрещает
безусловную петлю skip) */
  :: (gate == CLOSED && lamp == OFF) ->         /* турникет свободен */
       user_st = APPROACH;
       if
       :: m = CARD_OK  :: m = CARD_EMPTY
       :: m = TOKEN_OK :: m = TOKEN_BAD
       fi;
       if
       :: (m == CARD_OK  || m == CARD_EMPTY) -> card_in!m
       :: (m == TOKEN_OK || m == TOKEN_BAD)  -> token_in!m
```

```
       fi;
       user_st = WAIT;
       if
       :: (gate == OPEN) ->
            atomic { user_st = WALK; beam++; beam_hit = true; in_passage = true };
            atomic { beam--; in_passage = false; user_st = AWAY }
       :: (lamp == RED) ->                       /* отказ: забрать носитель */
            if
            :: (m == CARD_EMPTY) -> card_in!TAKE
            :: (m == TOKEN_BAD)  -> token_in!TAKE
            fi;
            user_st = AWAY
       fi
  od
}

/* «Паровозик»: проскакивает вплотную за оплатившим пассажиром */
active proctype Intruder() {
  do
  :: atomic { in_passage -> beam++; beam_hit = true; sneaked = true };
     atomic { beam--; sneaked = false };
     (!in_passage)
  od
}

/* ---------------------------- датчики ---------------------------- */
active proctype CardReader() {
  mtype m;
  do
  :: card_in?m ->
       if
       :: m == CARD_OK    -> card_res = OK;   to_ctrl!PAID
       :: m == CARD_EMPTY -> card_res = FAIL; to_ctrl!REJECTED
       :: m == TAKE       -> to_ctrl!TAKEN
       fi
  od
}

active proctype TokenAcceptor() {
  mtype m;
  do
  :: token_in?m ->
       if
       :: m == TOKEN_OK  -> token_res = OK;   to_ctrl!PAID
       :: m == TOKEN_BAD -> token_res = FAIL; to_ctrl!REJECTED
       :: m == TAKE      -> to_ctrl!TAKEN
       fi
  od
}

/* Фотоэлемент: импульс PASSED, когда луч перекрывался и проход освободился.
   Защёлка beam_hit нужна, чтобы не потерять короткое перекрытие. */
active proctype PassSensor() {
  do
  :: atomic { (beam_hit && beam == 0) -> beam_hit = false };
     to_ctrl!PASSED
  od
}

/* ------------------------- управление ------------------------- */
active proctype Controller() {
```

```
  mtype msg;
  do
  :: atomic { to_ctrl?msg; card_res = NONE; token_res = NONE };
     if
     :: msg == PAID ->
          ctrl_st = GRANTED;
          lamp_cmd!GREEN; gate_cmd!OPEN;
          to_ctrl?PASSED;                      /* ждём проход */
          ctrl_st = CLOSING;
          gate_cmd!CLOSED; (gate == CLOSED);
          lamp_cmd!OFF;
          ctrl_st = IDLE
     :: msg == REJECTED ->
          ctrl_st = DENIED;
          lamp_cmd!RED;
          to_ctrl?TAKEN;                       /* ждём, пока заберут носитель */
          lamp_cmd!OFF;
          ctrl_st = IDLE
     :: else -> skip                           /* посторонние события */
     fi
  od
}

/* -------------------------- актуаторы -------------------------- */
active proctype Gate() {
  mtype c;
  do :: gate_cmd?c -> gate = c od
}

active proctype Lamp() {
  mtype c;
  do :: lamp_cmd?c -> lamp = c od
}

/* -------------------------- свойства -------------------------- */
#ifdef P1  /* отказ => створки закрыты, пока не загорится красный */
ltl p1 { [] (rejected -> (gate == CLOSED U lamp == RED)) }
#endif
#ifdef P2  /* подтверждённая оплата => пассажир пройдёт */
ltl p2 { [] (paid -> <> (user_st == WALK)) }
#endif
#ifdef P3  /* открывшиеся створки открыты, пока луч не перекрыт (нужен -DNOREDUCE из-за X) */
ltl p3 { [] ((gate == CLOSED && X (gate == OPEN)) -> X (gate == OPEN U (beam > 0))) }
#endif
#ifdef P4  /* без оплаты пройти нельзя: ОЖИДАЕТСЯ НАРУШЕНИЕ */
ltl p4 { [] !sneaked }
#endif
/* Свидетели нетривиальности: нарушение = найден путь, где посылка
   выполняется бесконечно часто */
#ifdef V1
ltl v1 { !([] <> rejected) }
#endif
#ifdef V2
ltl v2 { !([] <> paid) }
#endif
#ifdef V3
ltl v3 { !([] <> (gate == CLOSED && X (gate == OPEN))) }
#endif
/* Система не застревает: контроллер бесконечно часто свободен */
#ifdef L1
```

```
ltl l1 { [] <> (ctrl_st == IDLE && gate == CLOSED) }
#endif
```